

Interface Gráfica, JTable e Depuração

Prof. Pedro A.B. Oliveira

FATEC Santana de Parnaíba

Curso: Análise e Desenvolvimento de Sistemas

Disciplina: Linguagem de Programação III - Desenvolver aplicações em JAVA

Aula 04 : 05 de Março de 2026

Conteúdo da Aula de Hoje

- Encontrar erros com o **Debug (Depuração)**.
- Os mistérios do pacote **javax.swing** e do **initComponents()**.
- A utilidade da anotação **@SuppressWarnings**.
- Conversão de tipos com **Casting**.
- Laços de repetição (**For Tradicional vs For-Each**).
- Manipulação de Tabelas (**DefaultTableModel** e **addRow**).
- Identificação de objetos com **instanceof**.

1. Depurando um Programa (Debug)

Como descobrir onde o seu cálculo salarial está a falhar

Passo a Passo no NetBeans:

- ➊ **Breakpoint:** Clique no número da linha para inserir um ponto de paragem (quadrado rosa). O Java vai pausar ali.
- ➋ **Iniciar:** Pressione Ctrl + Shift + F5 para rodar em modo Debug.

Controles Principais:

- **F8 (Step Over):** Executa a linha atual e pula para a próxima.
- **F7 (Step Into):** Entra dentro do método que está a ser chamado (ex: para ver a conta do INSS a acontecer).
- **Variáveis:** Passe o mouse por cima das variáveis para ver o valor exato na memória.

2. O Pacote `javax.swing`

Tudo o que é visual na nossa tela vem do pacote `javax.swing`.

- **O que é?** É a biblioteca padrão do Java para a criação de Interfaces Gráficas (GUI).
- **A Letra "J":** Todos os componentes começam com J (`JFrame`, `JButton`, `JTextField`, `JTable`).
- **Lightweight (Peso-leve):** Os componentes são desenhados pelo próprio Java. Isso significa que o seu botão terá a mesma aparência quer rode no Windows, Linux ou Mac.

3. O Método initComponents()

Já reparou naquele bloco de código cinzento que o NetBeans não o deixa apagar?

```
public TelaFuncionario() {  
    initComponents(); // 0 coracao da tela!  
}
```

- É um método gerado automaticamente pelo NetBeans.
- Ele lê o seu desenho (arrastar e largar componentes) e traduz para código Java real (new JButton(), configurar cores, tamanhos, e coordenadas X/Y).
- **Regra de Ouro:** Nunca tente editar o conteúdo deste método manualmente. Se precisar de esconder um botão ao abrir a tela, escreva a regra *depois* do initComponents();.

4. A anotação @SuppressWarnings

No Java, **Anotações** (começam com @) são recados para o compilador.

O @SuppressWarnings serve para silenciar alertas (linhas amarelas) da IDE. É como dizer: *"Eu sei o que estou a fazer, pare de me avisar"*.

```
@SuppressWarnings("unchecked")
public void preencherLista() {
    // Esconde o aviso de que o ArrayList nao tem um tipo <Generico
    >
    ArrayList listaAntiga = new ArrayList();
}
```

Atenção: Ele **não conserta erros**! Apenas varre os avisos para debaixo do tapete. Use apenas quando o código estiver garantidamente a funcionar.

5. O que é Casting?

Casting é o processo de forçar a conversão de um tipo para outro.

Com Tipos Primitivos (Números):

```
double valor = 10.99;  
int inteiro = (int) valor; // Resultado: 10 (perde as casas  
decimais)
```

Com Objetos (Herança e Polimorfismo): Permite tratar um objeto genérico como se fosse de uma classe mais específica (Downcasting).

```
Funcionario f = lista.get(0); // Pegamos um Funcionario comum  
// Forcamos o Java a trata-lo como Gerente para aceder ao bonus  
Gerente g = (Gerente) f;
```

6. A Classe DefaultTableModel

A JTable que desenhamos na tela é apenas uma "moldura". Ela não guarda os dados. Quem guarda a informação (o "cérebro" da tabela) é o **Model**.

```
// Pegamos a referencia do modelo da tabela e fazemos um Casting  
DefaultTableModel modelo = (DefaultTableModel) tbl.getModel();  
  
// Limpa todas as linhas para nao duplicar  
modelo.setNumRows(0);
```

Qualquer alteração feita na variável `modelo` (como adicionar ou apagar linhas) vai refletir-se instantaneamente na tela do usuário!

7 e 8. Os Tipos de Laços (Loops) no Java

O Java possui quatro estruturas principais para criar ciclos de repetição:

- 1. **while (Enquanto)**: Testa a condição *antes* de executar. Pode não rodar nenhuma vez.
- 2. **do-while (Faça-Enquanto)**: Executa o código e só testa a condição no fim. Garante que roda *pelo menos uma vez*.
- 3. **for Tradicional**: Usado quando sabemos exatamente quantas vezes queremos repetir (possui um contador, ex: `int i`).
- 4. **for-each (Avançado)**: Usado exclusivamente para percorrer coleções (como `ArrayList`) do início ao fim, focando no objeto.

7. For Tradicional vs For-Each

For Tradicional (Foco no Índice)

```
for (int i = 0; i < lista.size(); i++) {  
    Funcionario f = lista.get(i); // E preciso extrair o objeto  
    manualmente  
}
```

For-Each (Foco no Objeto)

Mais limpo, moderno e imune a erros de índice ("Array Out of Bounds").

```
for (Funcionario f : lista) {  
    // Leia: "Para cada Funcionario f dentro de lista..."  
    System.out.println(f.getNome());  
}
```

8. O Comando instanceof

Se a nossa lista está cheia de funcionários misturados, como o Java sabe se aquele objeto em específico é um Gerente ou um Horista?

O operador instanceof testa a identidade do objeto e devolve true ou false.

```
for (Funcionario f : lista) {  
    String cargo = "Comum";  
  
    // Testa se o objeto 'f' foi construído como Gerente  
    if (f instanceof Gerente) {  
        cargo = "Gerente"; // Altera o texto para a coluna da  
                           tabela  
    }  
}
```

9. O Método `addRow` e o `new Object[]`

Para criar uma linha na tabela, enviamos um **Vetor de Objetos**.

```
modelo.addRow(new Object[] {  
    f.getNome(),  
    cargo,  
    f.calcularLiquido()  
});
```

- `new Object[]`: Cria um contentor que aceita qualquer tipo de dado (texto, números, telas...).
- **Ordem**: Os itens separados por vírgula devem seguir a exata mesma ordem das colunas desenhadas no NetBeans (ex: Nome, Cargo, Salário).
- Graças ao **Polimorfismo**, o `f.calcularLiquido()` já executa a matemática correta de acordo com o cargo!

10. Imagens, Botões e Checkboxes na JTable

O comando `addRow(new Object[])` aceita imagens (`ImageIcon`) e botões (`JBButton`), mas por padrão, a tabela só exibe o endereço de memória (*toString*) deles.

Para a tabela mostrar os componentes reais (foto, caixa de seleção), é preciso usar ferramentas avançadas do Swing:

- **CellRenderer (Desenhador):** Ensina a coluna a ler o objeto e a "pintar" a imagem ou o botão na tela em vez de texto.
- **CellEditor (Editor):** Se quiser que o botão da tabela seja *clicável*, tem de configurar o Editor para captar os cliques do mouse naquela célula.

Dica: O NetBeans permite transformar uma coluna facilmente numa Checkbox se alterarmos o tipo da coluna nas Propriedades para `Boolean`!